



테스트 주도 개발방법론

박대우

강의 계획

주차	일자	강의 주제	강의 세부 내역
1 주차	2026-03-07	- 강의 Overview - TDD 개념	
2 주차	2026-03-14	소프트웨어 테스트 개념	
3 주차	2026-03-21	- 클린코드 - 클린코드를 위한 개발환경	
4 주차	2026-03-28	웹프런트 엔드 기초 & 부트스트랩	
5 주차	2026-04-04	Django 프로젝트 앱 개발하기	
6 주차	2026-04-11	정적파일과 미디어 파일 관리하기	
7 주차	2026-04-18	TDD 관련 사례 조사 발표	
8 주차	2026-04-25	중간 시험(또는 대체 과제)	

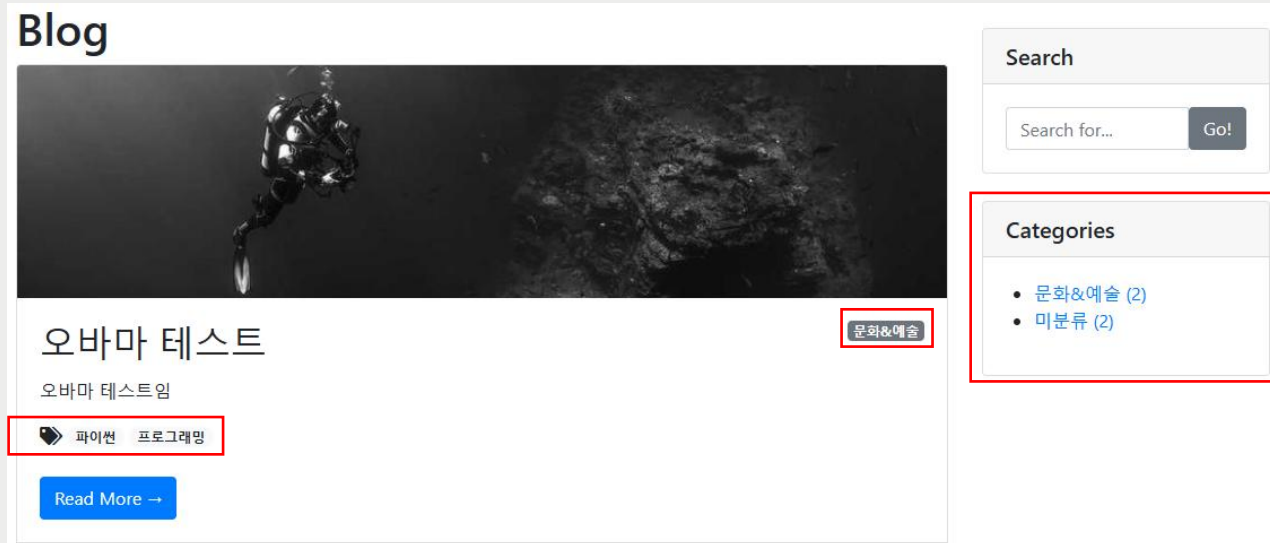
수업 개요

강의 계획

주차	일자	강의 주제	강의 세부 내역
9 주차	2026-05-02	▪ TDD로 템플릿 모듈화	
10 주차	2026-05-09	▪ 다대일 관계 구현	
11 주차	2026-05-16	▪ 다대다 관계 구현	
12 주차	2026-05-23	▪ Form으로 포스트 작성	
13 주차	2026-05-30	▪ 외부 라이브러리 활용	
14 주차	2026-06-06	▪ 대문페이지와 자기소개페이지 완성	
15 주차	2026-06-13	▪ 프로젝트 발표	

다대다 관계 구현

- 카테고리 기능 구현
- 다대다 관계 구현



1. 카테고리 기능 구현하기

□ Category 모델 만들기

- ✓ models.py에 Category 모델 포함하기
- ✓ SlugField: 숫자인 pk (primary key) 대신, 읽을 수 있는 텍스트로 URL을 만들고 싶을 때 주로 사용

실습 파일: blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
import os
```

```
class Category(models.Model):
```

```
    name = models.CharField(max_length=50, unique=True)
```

```
    slug = models.SlugField(max_length=200, unique=True, allow_unicode=True)
```

```
    def __str__(self):
```

```
        return self.name
```

(...생략...)

unique=True

유일한 값을 가져야 함

→ 이미 'python'이라는 category 레코드가 존재할 경우, 다른 레코드의 name 필드는 'python'이 될 수 없도록 강제

allow_unicode=True

원래 SlugField는 한글은 허용하지 않으나, allow=True를 설정하면 한글도 사용할 수 있음

1. 카테고리 기능 구현하기

□ Category 모델 만들기

- ✓ Post 모델에 category 필드 추가하기
- ✓ 카테고리가 미분류인 포스트도 있으니, null = True
- ✓ 카테고리가 삭제될 경우 연결된 포스트까지 삭제되지 않고, 해당 포스트의 카테고리만 null이 되게 함

```
class Post(models.Model):
    title = models.CharField(max_length=30)
    hook_text = models.CharField(max_length=100, blank=True)
    content = models.TextField()

    head_image = models.ImageField(upload_to='blog/images/%Y/%m/%d/', blank=True)
    file_upload = models.FileField(upload_to='blog/files/%Y/%m/%d/', blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    author = models.ForeignKey(User, null=True, on_delete=models.SET_NULL)

    category = models.ForeignKey(Category, null=True, on_delete=models.SET_NULL)
```

- ✓ 마이그레이션 수행: python manage.py makemigrations
- ✓ python manage.py migrate

1. 카테고리 기능 구현하기

□ Category 모델 만들기

- ✓ admin.py에 Category 모델 등록하기 : SlugField 자동으로 생성하는 admin 기능 만들기
- ✓ prepopulated_fields
 - name 필드 값으로 slug를 자동 생성하도록 설정

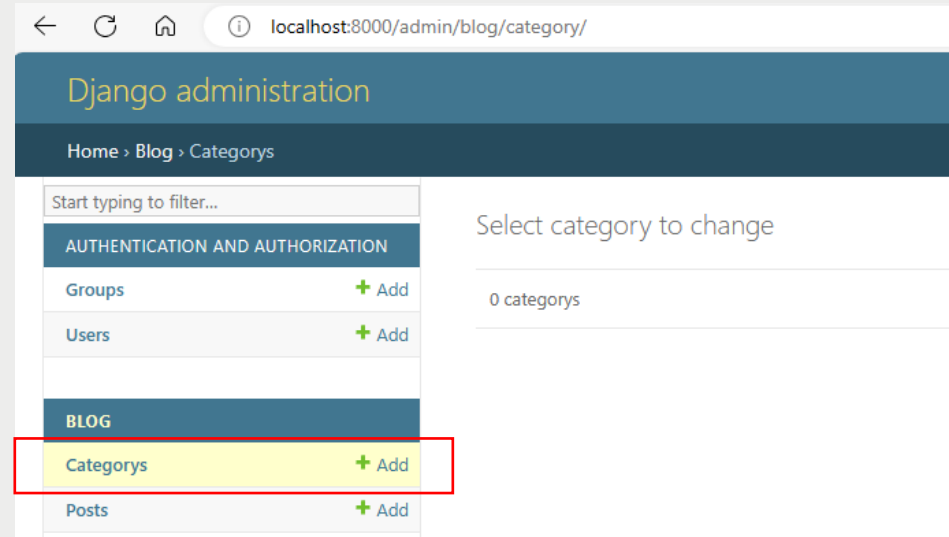
실습 파일: blog/admin.py

```
from django.contrib import admin
from .models import Post, Category

admin.site.register(Post)

class CategoryAdmin(admin.ModelAdmin):
    prepopulated_fields = {'slug': ('name', )}

admin.site.register(Category, CategoryAdmin)
```

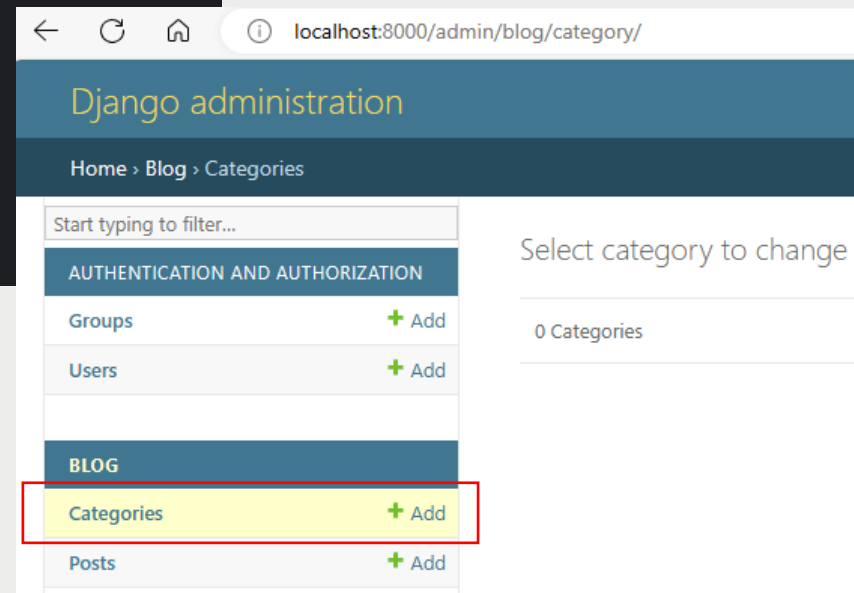


1. 카테고리 기능 구현하기

□ Category 모델 만들기

- ✓ Categorys → Categories
- ✓ models.py에 Category 모델의 메타 설정에서 verbose_name_plural을 추가해 복수형 지정

```
class Category(models.Model):  
    name = models.CharField(max_length=50, unique=True)  
    slug = models.SlugField(max_length=200, unique=True, allow_unicode=True)  
  
    new *  
    def __str__(self):  
        return self.name  
  
    new *  
    class Meta:  
        verbose_name_plural = 'Categories'
```



1. 카테고리 기능 구현하기

❑ Category를 빈칸으로 남겨두기

✓ 카테고리 하나 만들기

✓ 카테고리 없이 저장시 오류발생

Django administration

Home > Blog > Categories > Add category

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

BLOG

Categories + Add

Posts + Add

Add category

Name: 문화 & 예술

Slug: 문화-예술

SAVE Save and add another Save and continue editing

Change post

HISTORY VIEW ON SITE >

[3]피고인의 자백 :: pdw90

Please correct the error below.

Title: 피고인의 자백

Hook text:

Content:

피고인의 자백이 고문·폭행·협박·구속의 부당한 장기화 또는 기망 기타의 방법에 의하여 자의로 진술된 것이 아니라고 인정될 때 또는 정식재판에 있어서 피고인의 자백이 그에게 불리한 유일한 증거일 때에는 이를 유죄의 증거로 삼거나 이를 이유로 처벌할 수 없다. 대통령의 임기는 5년으로 하며, 중임할 수 없다.

국무위원은 국무총리의 제청으로 대통령이 임명한다. 국가는 농지에 관하여 경자유전의 원칙이 달성될 수 있도록 노력하여야 하며, 농지의 소작제도는 금지된다. 국군은 국가의 안전보장과 국토방위의 신성한 의무를 수행함을 사명으로 하며, 그 정치적 중립성은 준수된다. 헌경권의 내용과 행사에 관하여는 법률로 정한다.

Head image: 파일 선택 선택된 파일 없음

File upload: 파일 선택 선택된 파일 없음

Author: pdw90 + - x

Category: This field is required. + - x

SAVE Save and add another Save and continue editing Delete

1. 카테고리 기능 구현하기

❑ Category를 빈칸으로 남겨두기

- ✓ Post 모델의 category 필드 공란 허용하기

null=True

DB 상에 공란으로 저장될 수 있도록 허용

blank=True

admin이나 form에서 유효성
검사 할 때, 공란 허용

실습 파일: blog/models.py

(...생략...)

```
category = models.ForeignKey(Category, null=True, blank=True, on_delete=models.SET_NULL)
```

(...생략...)

- ✓ 마이그레이션 수행
- ✓ admin 페이지에서 카테고리 없이 저장되는지 확인

1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

✓ 테스트 코드 작성하기

- 카테고리 2개 생성 & 포스트도 setUp 함수로 이동(self. 붙여서)

```
tests.py ×
1 from django.test import TestCase, Client
2 from bs4 import BeautifulSoup
3 from .models import Post, Category
4 from django.contrib.auth.models import User
5
6
7 class TestView(TestCase):
8     def setUp(self):
9         self.client = Client()
10        self.user_trump = User.objects.create_user(
11            username="trump", password="somepassword"
12        )
13        self.user_obaba = User.objects.create_user(
14            username="obama", password="somepassword"
15        )
16
17        self.category_programming = Category.objects.create(
18            name="programming", slug="programming"
19        )
20        self.category_music = Category.objects.create(
21            name="music", slug="music"
22        )
```

```
self.post_001 = Post.objects.create(
    title="첫번째 포스트입니다.",
    content="Hello World. We are the world.",
    category=self.category_programming,
    author=self.user_trump,
)

self.post_002 = Post.objects.create(
    title="두번째 포스트입니다.",
    content="1등이 전부는 아니잖아요?",
    category=self.category_music,
    author=self.user_obaba,
)

self.post_003 = Post.objects.create(
    title="세번째 포스트입니다.",
    content="Category가 없어요~~",
    author=self.user_trump,
)
```

1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

✓ category_card_test() 함수 따로 만들기

```
tests.py ×
12 def category_card_test(self, soup):
13     categories_card = soup.find("div", id="categories-card")
14     self.assertIn(member: "Categories", categories_card.text)
15     self.assertIn(
16         member: f'{self.category_programming.name} ({self.category_programming.post_set.count()})',
17         categories_card.text,
18     )
19     self.assertIn(
20         member: f'{self.category_music.name} ({self.category_music.post_set.count()})',
21         categories_card.text,
22     )
23     self.assertIn(member: f'미분류 (1)', categories_card.text)
```

1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

✓ test_post_list() 함수 수정하기: post가 있는 경우와 없는 경우로 나눠서 테스트

```
def test_post_list(self):
    # Post가 있는 경우
    self.assertEqual(Post.objects.count(), second: 3)

    response = self.client.get("/blog/")
    self.assertEqual(response.status_code, second: 200)
    soup = BeautifulSoup(response.content, features: "html.parser")

    self.navbar_test(soup)
    self.category_card_test(soup)

    main_area = soup.find(name: "div", id="main-area")
    self.assertNotIn(member: "아직 게시물이 없습니다", main_area.text)

    # id가 post-1인 div를 찾아서, 그 안에
    post_001_card = main_area.find("div", id="post-1")
    # title이 있는지
    self.assertIn(self.post_001.title, post_001_card.text)
    # category가 있는지
    self.assertIn(self.post_001.category.name, post_001_card.text)
```

```
post_002_card = main_area.find("div", id="post-2")
self.assertIn(self.post_002.title, post_002_card.text)
self.assertIn(self.post_002.category.name, post_002_card.text)

post_003_card = main_area.find("div", id="post-3")
self.assertIn(member: "미분류", post_003_card.text)
self.assertIn(self.post_003.title, post_003_card.text)

# Post가 없는 경우
Post.objects.all().delete()
self.assertEqual(Post.objects.count(), second: 0)
response = self.client.get("/blog/")
soup = BeautifulSoup(response.content, features: "html.parser")
# id가 main-area인 div태그를 찾습니다.
main_area = soup.find(name: "div", id="main-area")
self.assertIn(member: "아직 게시물이 없습니다", main_area.text)
```

✓ test 수행: python manage.py test blog.tests.TestView.test_post_list

```
File "D:\work\python\project\mysite\blog\tests.py", line 44, in category_card_test
    self.assertIn("Categories", categories_card.text)
AttributeError: 'NoneType' object has no attribute 'text'
```

1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

- ✓ base.html에서 카테고리에 해당하는 div 요소에 id='categories-card'를 추가

```
<!-- Categories Widget -->
<div class="card my-4" id="categories-card">
  <h5 class="card-header">Categories</h5>
  <div class="card-body">
    <div class="row">
      <div class="col-lg-6">
        <ul class="list-unstyled mb-0">
          <li>
            <a href="#">Web Design</a>
          </li>
          <li>
            <a href="#">HTML</a>
          </li>
          <li>
            <a href="#">Freebies</a>
          </li>
        </ul>
      </div>
    </div>
  </div>
</div>
```

아래부분 모두 삭제

```
<div class="col-lg-6">
  <ul class="list-unstyled mb-0">
    <li>
      <a href="#">JavaScript</a>
    </li>
    <li>
      <a href="#">CSS</a>
    </li>
    <li>
      <a href="#">Tutorials</a>
    </li>
  </ul>
</div>
```

- ✓ test 수행: python manage.py test blog.tests.TestView.test_post_list

```
AssertionError: 'programming (1)' not found in '\nCategories\n\n\n\n\n\nWeb Design\n\n\nHTML\n\n\nFreebies\n\n\n\n\n\n'
```

1. 카테고리 기능 구현하기

❑ 포스트 목록페이지 수정하기: `get_context_data` & `모델명.objects.filter(조건)`

✓ CBV에서 추가로 넘기고 싶은 인자가 있을 때 사용: category 목록 가져와서 딕셔너리로 저장

실습 파일: `blog/views.py`

```
from django.shortcuts import render
from django.views.generic import ListView, DetailView
from .models import Post, Category
```

```
class PostList(ListView):
```

```
    model = Post
```

```
    ordering = '-pk'
```

```
    def get_context_data(self, **kwargs):
```

```
        context = super(PostList, self).get_context_data()
```

```
        context['categories'] = Category.objects.all()
```

```
        context['no_category_post_count'] = Post.objects.filter(category=None).
```

```
count()
```

```
        return context
```

```
class PostDetail(DetailView):
```

```
    model = Post
```

CBV는 `get_context_data` 함수를 내장하고 있으므로, 오버라이딩 한 후, 추가하고 싶은 요소를 context 딕셔너리에 담아 템플릿으로 보낼 수 있음
model = Post를 선언하면 `post_list = Post.object.all()`를 내부적으로 수행

`Post.objects.all()`과 달리, 특정 조건인 레코드만 필터링 할 때 사용
→ `category=None`인 Post만 필터링

1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

✓ 템플릿 수정하기: base.html 수정

```
<!-- Categories Widget -->
<div class="card my-4" id="categories-card">
  <h5 class="card-header">Categories</h5>
  <div class="card-body">
    <div class="row">
      <ul>
        {% for category in categories %}
          <li>
            <a href="{% category.get_absolute_url %}">{% category %} ({{ category.post_set.count }})</a>
          </li>
        {% endfor %}
        <li>
          <a href="/blog/category/no_category/">미분류 ({{ no_category_post_count }})</a>
        </li>
      </ul>
    </div>
  </div>
</div>
```

✓ test 수행: `python manage.py test blog.tests.TestView.test_post_list` → `post_001_card`가 없음

```
Traceback (most recent call last):
  File "D:\work\python\project\mysite\blog\tests.py", line 89, in test_post_list
    self.assertIn(self.post_001.title, post_001_card.text)
AttributeError: 'NoneType' object has no attribute 'text'
```


1. 카테고리 기능 구현하기

□ 포스트 목록페이지 수정하기

✓ 템플릿 수정하기: post_list.html 수정 - id="post-{{p.pk}}" 추가 후 테스트 수행

post_list.html

```
<!-- Blog Post -->
<div class="card mb-4" id="post-{{ p.pk }}">
    {% if p.head_image %}
```

```
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 91, in test_post_list
    self.assertIn(self.post_001.category.name, post_001_card.text)
AssertionError: 'programming' not found in '\n\n\n첫 번째 포스트입니다.\n\n
Posted on May 15, 2024, 1:31 p.m. by\n'
```

✓ 카테고리 추가 후 테스트 → 브라우저에서 확인

post_list.html

```
<div class="card-body">
    {% if p.category %}
        <span class="badge badge-secondary float-right">{{ p.category }}</span>
    {% else %}
        <span class="badge badge-secondary float-right">미분류</span>
    {% endif %}
    <h2 class="card-title">{{ p.title }}</h2>
```

1. 카테고리 기능 구현하기

□ 포스트 상세페이지 수정하기

✓ 테스트 코드 수정하기: post_001 → self.post_001로 수정(tests.py)

```
def test_post_detail(self):
    # 0.1 그 포스트의 url은 'blog/1/' 이다.
    self.assertEqual(self.post_001.get_absolute_url(), second: "/blog/1/")

    # 1. 첫 번째 post의 detail 페이지 테스트
    # 1.1 첫 번째 post url로 접근하면 정상적으로 작동한다. (status code: 200)
    response = self.client.get(self.post_001.get_absolute_url())
    self.assertEqual(response.status_code, second: 200)
    soup = BeautifulSoup(response.content, features: "html.parser")

    # 1.2 post_list 페이지와 똑같은 네비게이션 바가 있다.
    self.navbar_test(soup)
    self.category_card_test(soup)

    # 1.3 첫 번째 post의 title이 브라우저 탭에 표시되는 페이지 title에 있다.
    self.assertIn(self.post_001.title, soup.title.text)
```

tests.py

```
# 1.4 첫 번째 post의 title이 post-area에 있다.
main_area = soup.find( name: "div", id="main-area")
post_area = main_area.find("div", id="post-area")
self.assertIn(self.post_001.title, post_area.text)
self.assertIn(self.category_programming.name, post_area.text)

# 1.5 첫 번째 post의 작성자(author)가 post-area에 있다.
self.assertIn(self.user_trump.username.upper(), main_area.text)

# 1.6 첫 번째 post의 content가 post-area에 있다.
self.assertIn(self.post_001.content, post_area.text)
```

✓ 테스트 수행: python manage.py test

```
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 125, in test_post_detail
    self.category_card_test(soup)
  File "C:\work\python\mysite\blog\tests.py", line 45, in category_card_test
    self.assertIn(
AssertionError: 'programming (1)' not found in '\nCategories\n\n\n\n\n미분류 ()\n\n\n\n\n'
```

1. 카테고리 기능 구현하기

□ 포스트 상세페이지 수정하기

✓ get_context_data() 메소드로 category 관련 인자 넘기기

views.py

```
class PostDetail(DetailView):
    model = Post

    new *
    def get_context_data(self, **kwargs):
        context = super(PostDetail, self).get_context_data()
        context["categories"] = Category.objects.all()
        context["no_category_post_count"] = Post.objects.filter(category=None).count()
        return context
```

✓ post_detail.html 수정 후 테스트 수행

```
<div id="post-area">
    {% if post.category %}
        <span class="badge badge-secondary float-right">{{ post.category }}</span>
    {% else %}
        <span class="badge badge-secondary float-right">미분류</span>
    {% endif %}
<!-- Title -->
<h1 class="mt-4">{{ post.title }}</h1>
<h5 class="text_muted">{{ post.hook_text }}</h5>
```

1. 카테고리 기능 구현하기

□ 카테고리 페이지 만들기

✓ 테스트 코드 작성하기: 한 카테고리를 클릭하면 해당 카테고리와 관계있는 포스트만 보여주기 `tests.py`

```
def test_category_page(self):
    response = self.client.get(self.category_programming.get_absolute_url())
    self.assertEqual(response.status_code, 200)

    soup = BeautifulSoup(response.content, features='html.parser')
    self.navbar_test(soup)
    self.category_card_test(soup)

    self.assertIn(self.category_programming.name, soup.h1.text)

    main_area = soup.find(name='div', id='main-area')
    self.assertIn(self.category_programming.name, main_area.text)
    self.assertIn(self.post_001.title, main_area.text)
    self.assertNotIn(self.post_002.title, main_area.text)
    self.assertNotIn(self.post_003.title, main_area.text)
```

✓ 테스트 수행하면 오류 발생

1. 카테고리 기능 구현하기

□ 카테고리 페이지 만들기

✓ Category 모델 수정하기, URL 정의하기(blog/urls.py 수정)

models.py

```
class Category(models.Model):
    name = models.CharField(max_length=50, unique=True)
    slug = models.SlugField(max_length=200, unique=True, allow_unicode=True)

    new *
    def __str__(self):
        return self.name

    3 usages (3 dynamic) new *
    def get_absolute_url(self):
        return f'/blog/category/{self.slug}/'

    new *
    class Meta:
        verbose_name_plural = 'Categories'
```

urls.py

```
urlpatterns = [
    path('category/<str:slug>/', views.category_page),
    path('', views.PostList.as_view()),
    path('<int:pk>/', views.PostDetail.as_view()),
]
```

1. 카테고리 기능 구현하기

□ 카테고리 페이지 만들기

✓ 뷰 수정하기

views.py

```
def category_page(request, slug):
    category = Category.objects.get(slug=slug)

    return render(
        request,
        template_name: 'blog/post_list.html',
        context: {
            'post_list': Post.objects.filter(category=category),
            'categories': Category.objects.all(),
            'no_category_post_count': Post.objects.filter(category=None).count(),
            'category': category,
        }
    )
```

✓ 테스트 수행 > 오류발생

```
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 151, in test_category_page
    self.assertIn(self.category_programming.name, soup.h1.text)
AssertionError: 'programming' not found in 'Blog'
```

1. 카테고리 기능 구현하기

□ 카테고리 페이지 만들기

✓ post_list.html 수정 → 카테고리가 있을 경우는 정상동작, 미분류 클릭하면 오류 발생

post_list.html

```
{% block main_area %}
    <h1>Blog {% if category %}<span class="badge badge-secondary">{{ category }}</span>
    {% endif %}</h1>

    {% if post_list.exists %}
```

✓ 미분류일 경우 처리: views.py

```
def category_page(request, slug):
    if slug == 'no_category':
        category = '미분류'
        post_list = Post.objects.filter(category=None)
    else:
        category = Category.objects.get(slug=slug)
        post_list = Post.objects.filter(category=category)

    return render(
        request,
        template_name: 'blog/post_list.html',
        context: {
            'post_list': post_list,
            'categories': Category.objects.all(),
            'no_category_post_count': Post.objects.filter(category=None).count(),
            'category': category,
        }
    )
```

✓ 웹 브라우저에서 확인

실습

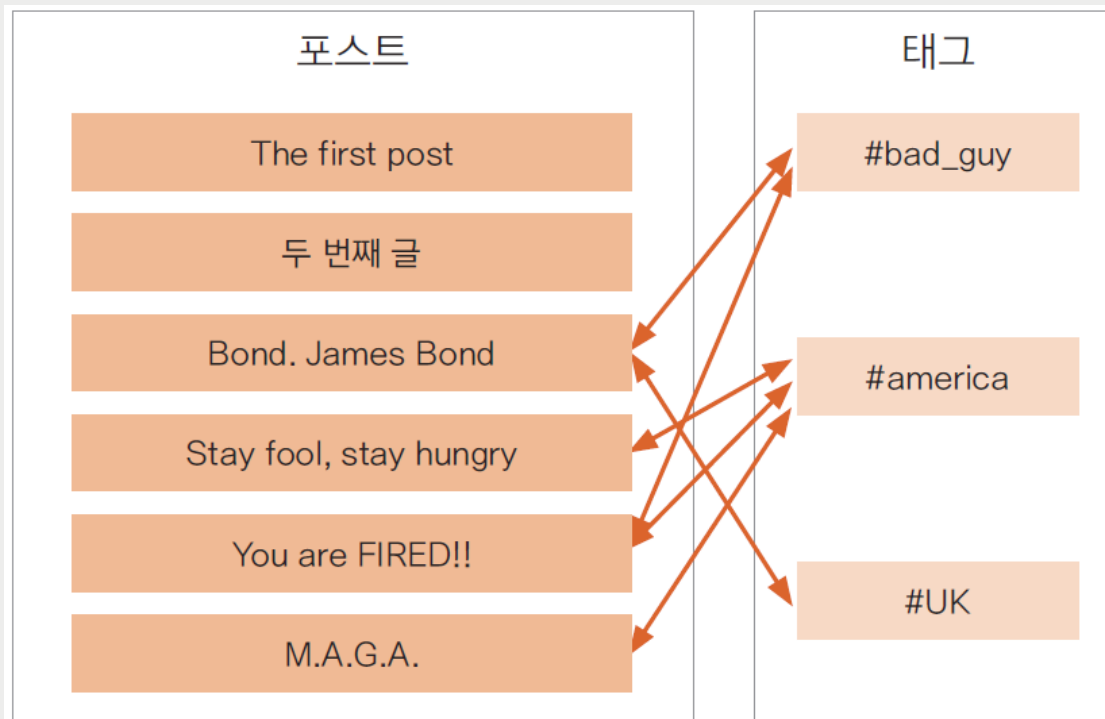


- ❑ Category 모델 만들기
 - ✓ Category 빈칸이 들어가도록 실습
- ❑ 포스트 목록페이지 수정하기
- ❑ 포스트 상세페이지 수정하기
- ❑ 카테고리 페이지 만들어보기

1. 다대다 관계(many to many relation)

□ A모델의 여러 레코드가 B 모델의 여러 레코드에 연결 될 수 있는 관계

✓ 예시: Post 모델의 레코드는 Tag 모델의 여러 레코드와 연결될 수 있으며, Tag 모델의 레코드도 여러 Post 모델의 레코드와 연결될 수 있음



2. Tag 모델 만들기

□ Tag 모델 구현하기

- ✓ Tag 모델 작성하기: category 모델과 비슷
- ✓ Post 모델에 tag 추가

실습 파일: blog/models.py

```
from django.db import models
from django.contrib.auth.models import User
import os

class Tag(models.Model):
    name = models.CharField(max_length=50)
    slug = models.SlugField(max_length=200, unique=True, allow_unicode=True)

    def __str__(self):
        return self.name

    def get_absolute_url(self):
        return f'/blog/tag/{self.slug}/'

(...생략...)
```

ManyToManyField는 null=True를 설정할 필요 없음

실습 파일: blog/models.py

```
(...생략...)
class Post(models.Model):
    (...생략...)
    tags = models.ManyToManyField(Tag, blank=True)
    (...생략...)
```

2. Tag 모델 만들기

□ Tag 모델 구현하기

✓ 관리자 페이지에 태그 추가하기

admin.py

```
from django.contrib import admin
from .models import Post, Category, Tag

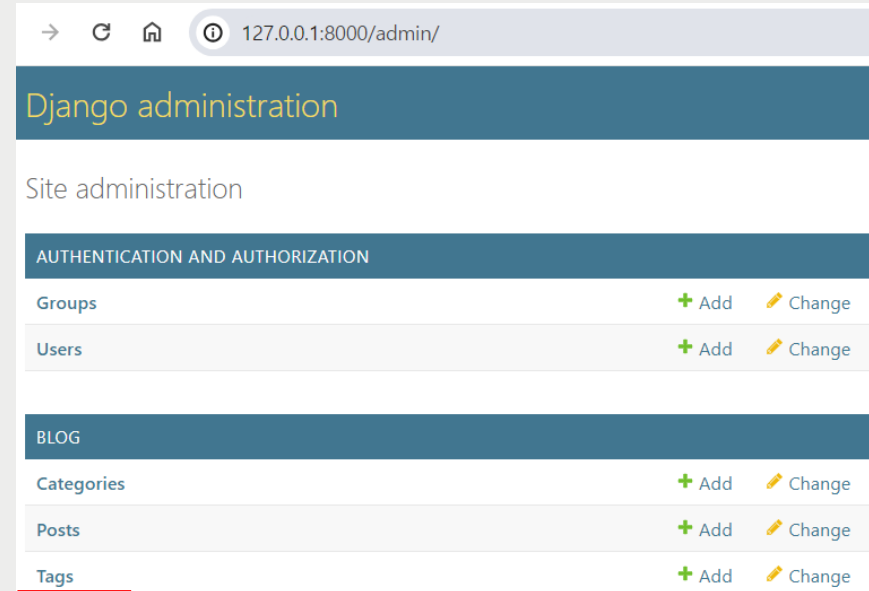
admin.site.register(Post)

1 usage new *
class CategoryAdmin(admin.ModelAdmin):
    prepopulated_fields = {'slug': ('name', )}

1 usage new *
class TagAdmin(admin.ModelAdmin):
    prepopulated_fields = {'slug': ('name', )}

admin.site.register(Category, CategoryAdmin)
admin.site.register(Tag, TagAdmin)
```

→ python manage.py makemigrations
python manage.py migrate



3. 포스트 모델과 상세 페이지에 태그 기능 추가하기

□ 테스트 코드에 태그 추가하기

✓ tests.py에 태그 추가하기, 태그 3개를 포스트에 연결하기

```
from .models import Post, Category, Tag
from django.contrib.auth.models import User
```

pdw90 *

```
class TestView(TestCase):
```

pdw90 *

```
def setUp(self):
```

```
    self.client = Client()
```

```
    self.user_trump = User.objects.create_user(
```

```
        username="trump", password="somepassword"
```

```
)
```

```
    self.user_obama = User.objects.create_user(
```

```
        username="obama", password="somepassword"
```

```
)
```

```
    self.category_programming = Category.objects.create(
```

```
        name="programming", slug="programming"
```

```
)
```

```
    self.category_music = Category.objects.create(name="music", slug="music")
```

```
    self.tag_python_kor = Tag.objects.create(name="파이썬 공부", slug='파이썬-공부')
```

```
    self.tag_python = Tag.objects.create(name="python", slug='python')
```

```
    self.tag_hello = Tag.objects.create(name="hello", slug='hello')
```

```
self.post_003 = Post.objects.create(
    title="세번째 포스트입니다.",
    content="Category가 없어요~~",
    author=self.user_trump,
)
```

```
self.post_001.tags.add(self.tag_hello)
self.post_003.tags.add(self.tag_python_kor)
self.post_003.tags.add(self.tag_python)
```

3. 포스트 모델과 상세 페이지에 태그 기능 추가하기

□ 포스트 목록페이지에 태그기능 추가하기

✓ 테스트코드 수정하기

✓ 테스트수행 →
오류 발생

tests.py

```
post_001_card = main_area.find("div", id="post-1")
self.assertIn(self.post_001.title, post_001_card.text)
self.assertIn(self.post_001.category.name, post_001_card.text)
self.assertIn(self.post_001.author.username.upper(), post_001_card.text)
self.assertIn(self.tag_hello.name, post_001_card.text)
self.assertNotIn(self.tag_python.name, post_001_card.text)
self.assertNotIn(self.tag_python_kor.name, post_001_card.text)
```

```
post_002_card = main_area.find("div", id="post-2")
self.assertIn(self.post_002.title, post_002_card.text)
self.assertIn(self.post_002.category.name, post_002_card.text)
self.assertIn(self.post_002.author.username.upper(), post_002_card.text)
self.assertNotIn(self.tag_hello.name, post_002_card.text)
self.assertNotIn(self.tag_python.name, post_002_card.text)
self.assertNotIn(self.tag_python_kor.name, post_002_card.text)
```

```
post_003_card = main_area.find("div", id="post-3")
self.assertIn(member: "미분류", post_003_card.text)
self.assertIn(self.post_003.title, post_003_card.text)
self.assertIn(self.post_003.author.username.upper(), post_003_card.text)
self.assertNotIn(self.tag_hello.name, post_003_card.text)
self.assertIn(self.tag_python.name, post_003_card.text)
self.assertIn(self.tag_python_kor.name, post_003_card.text)
```

3. 포스트 모델과 상세 페이지에 태그 기능 추가하기

□ 포스트 목록페이지에 태그기능 추가하기

✓ post_list.html 수정하기

post_list.html

```
{% if p.tags.exists %}
    <i class="fas fa-tags"></i>
    {% for tag in p.tags.iterator %}
        <a href="{{ tag.get_absolute_url }}">
            <span class="badge badge-pill badge-light">{{ tag }}</span></a>
    {% endfor %}
    <br/>
    <br/>
{% endif %}
```

```
<a href="{{ p.get_absolute_url }}" class="btn btn-primary">Read More </a>
```

✓ 태그 모양 아이콘은 Font awesome 사이트에서 가져옴, 태그 모양과 색상은 부트스트랩에서 가져옴

✓ 테스트수행 → 성공

3. 포스트 모델과 상세 페이지에 태그 기능 추가하기

□ 포스트 상세페이지에 태그기능 추가하기

- ✓ 테스트 코드 수정하기: tests.py의 test_post_detail 함수의 맨 아래

```
# 1.6 첫 번째 post의 content가 post-area에 있다.  
self.assertIn(self.post_001.content, post_area.text)  
  
self.assertIn(self.tag_hello.name, post_area.text)  
self.assertNotIn(self.tag_python.name, post_area.text)  
self.assertNotIn(self.tag_python_kor.name, post_area.text)
```

- ✓ post_detail.html 수정

- ✓ 테스트 수행 →
성공

```
<!-- Post Content -->  
<p>{{ post.content }}</p>  
  
{% if post.tags.exists %}  
  <i class="fas fa-tags"></i>  
  {% for tag in post.tags.all %}  
    <a href="{{ tag.get_absolute_url }}">  
      <span class="badge badge-pill badge-light">{{ tag }}</span></a>  
  {% endfor %}  
  <br/>  
  <br/>  
{% endif %}  
  
{% if post.file_upload %}
```

4. 태그 페이지 만들기

□ 태그 페이지 만들기

✓ 테스트 코드 작성하기: tests.py

```
def test_tag_page(self):
    response = self.client.get(self.tag_hello.get_absolute_url())
    self.assertEqual(response.status_code, 200)
    soup = BeautifulSoup(response.content, features='html.parser')

    self.navbar_test(soup)
    self.category_card_test(soup)

    self.assertIn(self.tag_hello.name, soup.h1.text)

    main_area = soup.find(name='div', id='main-area')
    self.assertIn(self.tag_hello.name, main_area.text)
    self.assertIn(self.post_001.title, main_area.text)
    self.assertNotIn(self.post_002.title, main_area.text)
    self.assertNotIn(self.post_003.title, main_area.text)
```

✓ 테스트 수행

```
Traceback (most recent call last):
  File "C:\work\python\mysite\blog\tests.py", line 53, in test_tag_page
    self.assertEqual(response.status_code, 200)
AssertionError: 404 != 200
```


4. 태그 페이지 만들기

□ 태그 페이지 만들기

✓ url 지정하기

urls.py

```
urlpatterns = [  
    path('tag/<str:slug>/', views.tag_page),  
    path('category/<str:slug>/', views.category_page),  
    path('', views.PostList.as_view()),  
    path('<int:pk>/', views.PostDetail.as_view()),  
]
```

✓ views.py 수정 → 테스트 수행하면 오류 발생

views.py

```
def tag_page(request, slug):  
    tag = Tag.objects.get(slug=slug)  
    post_list = tag.post_set.all()  
  
    return render(  
        request,  
        template_name='blog/post_list.html',  
        context={  
            'post_list': post_list,  
            'tag': tag,  
            'categories': Category.objects.all(),  
            'no_category_post_count': Post.objects.filter(category=None).count(),  
        })
```

4. 태그 페이지 만들기

□ 태그 페이지 만들기

✓ post_list.html 수정하기

```
{% block main_area %}
    <h1>Blog {% if category %}<span class="badge badge-secondary">{{ category }}</span>
    {% endif %}
    <span class="badge badge-secondary">{{ tag }}</span>
    {% endif %}
</h1>
```

✓ 테스트 성공: 태그페이지 클릭해쓸 경우 tag인지 카테고리인지 구분 안됨

Blog **python**

✓ 아이콘과 건수 추가

```
<h1>Blog {% if category %}<span class="badge badge-secondary">{{ category }}</span>
{% endif %}
{% if tag %}<span class="badge badge-light"><i class="fas fa-tags"></i>
        {{ tag }} ({{tag.post_set.count }})</span>
{% endif %}
</h1>
```

Blog  hello (2)

실습



- ❑ Tag 모델 만들기
- ❑ 포스트 목록페이지와 포스트 상세페이지에 태그기능 추가하기
- ❑ 태그 페이지 만들어보기

감사합니다.